

Databases 2 - exam - January 15, 2025 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency control (11 points)

Consider the following *arrival sequence* AS , decorated with lock and unlock requests:

$$AS = S_1(b) S_1(c) r_1(b) X_2(b) w_2(b) X_2(a) r_2(a) w_2(a) r_1(c) U_1(c) U_1(b) X_2(c) r_2(c) w_2(c) U_2(c) U_2(b) U_2(a) S_1(a) r_1(a) U_1(a),$$

where $S_i(z)$ indicates a shared lock request by transaction i on resource z , and similarly for $X_i(z)$ (exclusive lock) and $U_i(z)$ (unlock). Let H be the *history* or *a-posteriori sequence* (without the indication of lock and unlock requests) derived from AS by a scheduler that grants locks whenever possible, or else puts the requesting transaction on hold until the requested resource becomes free. Every request in AS must be processed, regardless of whether the transaction issuing the request is currently waiting on other resources.

1. Show H . Is H guaranteed to avoid all anomalies? Explain. [2 points]
2. Classify H with respect to VSR, CSR, 2PL, Strict 2PL, TS Mono, and TS Multi (with the conventions adopted for TS Multi under Snapshot Isolation, used for the exercises). Justify your answers concisely (determine membership via class inclusion wherever possible). [up to 1.5 points per class]

Solution.

1. The resulting history $H = r_1(b)r_2(a)w_2(a)r_1(c)w_2(b)r_2(c)w_2(c)r_1(a)$ differs from AS only by the position of $w_2(b)$ and is not anomaly-free. The reason for anomalies to be possible is that respecting the lock assignment policy alone is not sufficient for granting serializability: the original arrival sequence did not respect the 2PL policy from the start, with transaction 1 releasing the lock on c before acquiring it on a .
2. It is easy to check that H is not in VSR, since there are only two transactions: serial schedule $T_1 - T_2$ is not view-equivalent to H because $r_1(a)$ would no longer read from $w_2(a)$; the other serial schedule, $T_2 - T_1$, is also not view-equivalent to H because $r_1(b)$ would no longer read from the database state. By using class inclusion, we conclude that the schedule is outside CSR, 2PL, Strict 2PL, and TS Mono. Alternatively, one can first check that H is not in CSR (the conflict graph is cyclic due to, e.g., $r_1(c) \rightarrow w_2(c)$ and $w_2(a) \rightarrow r_1(a)$) and conclude that H is not in VSR because there are no inactive writes.

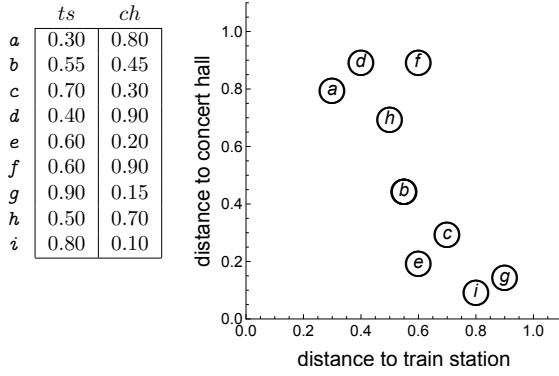
However, H is in TS Multi:

Operation	RTM(a)	WTM(a)	RTM(b)	WTM(b)	RTM(c)	WTM(c)	Killed transactions
$r_1(b)$	0 (0)	0	1 (0)	0	0 (0)	0	
$r_2(a)$	2 (0)						
$w_2(a)$		0, 2					
$r_1(c)$					1 (0)		
$w_2(b)$				0, 2			
$r_2(c)$					2 (0)		
$w_2(c)$						0, 2	
$r_1(a)$	2 (0)						

Remark: If for the waiting transaction, its operations are not processed, the resulting history is: $H = r_1(b)r_1(c)w_2(b)r_2(a)w_2(a)r_2(c)w_2(c)r_1(a)$. This schedule is not in VSR, not in CSR, not in 2PL, not in strict 2PL, not in TS Mono. It is in TS Multi.

B. Ranking (10 points)

Consider the following dataset D of hotels with their distances to points of interest (train station ts and concert hall ch).



1. Compute the skyline of D with SFS. Indicate all dominance tests. [4 points]
2. Indicate the 2-skyband of D . [2 points]
3. Which hotels can be top-1 according to functions in L_1 (i.e., linear scoring functions)? [2 points]
4. Let $\mathcal{F}$ be the set of functions of the form $f(x, y) = w_1x + w_2y$, where $w_1 > w_2$, i.e., linear combinations giving more importance to the train station. Which hotels can be top-1 according to some function in $\mathcal{F}$? [2 points]

Solution.

1. Sorting the dataset by first ts then ch gives:

	ts	ch
a	0.30	0.80
d	0.40	0.90
h	0.50	0.70
b	0.55	0.45
e	0.60	0.20
f	0.60	0.90
c	0.70	0.30
i	0.80	0.10
g	0.90	0.15

Then: $a < d$, $a \not< h$, $a \not< b$, $h \not< b$, $a \not< e$, $h \not< e$, $b \not< e$, $a < f$, $a \not< c$, $h \not< c$, $b \not< c$, $e < c$, $a \not< i$, $h \not< i$, $b \not< i$, $e \not< i$, $a \not< g$, $h \not< g$, $b \not< g$, $e \not< g$, $i < g$. So the skyline is $\{a, h, b, e, i\}$.

2. The 2-skyband is the set of tuples dominated by less than 2 tuples, so it includes all skyline tuples, plus d , c , g . The only tuple not included is f , which is dominated by 5 tuples.
3. All skyline tuples are top-1 for some monotone scoring function. However, if we restrict to linear scoring functions, then only those lying on the convex hull qualify: a , e , i .
4. If we further restrict to functions in $\mathcal{F}$, then i must also be excluded, since its score $w_1 0.8 + w_2 0.1$ is always going to be worse than e 's score $w_1 0.6 + w_2 0.2$: indeed, $w_1 0.8 + w_2 0.1 \geq w_1 0.6 + w_2 0.2$ entails $w_1 0.2 \geq w_2 0.1$, which always holds when $w_1 > w_2$. So, only a and e can be top-1.

C. Physical DB (11 points)

A bus travel system includes the following tables: **Ticket** (**TicketId**, **RouteFrom**, **RouteTo**, **PassengerId**, **TravelDate**, **DepartureTime**, **ArrivalTime**) containing 300K tickets in 45K blocks in an entry-sequenced primary storage. Table **Passenger**(**PassengerId**, **Name**, **Email**, **City**) stores 40K tuples in a B+ tree built on **PassengerId** with 3 levels and 2.5K leaf nodes. $\text{val}(\text{RouteTo}) = 250$ and $\text{val}(\text{City}) = 500$. Evaluate the cost in terms of I/O accesses of the following query

```
SELECT PassengerId
FROM Ticket NATURAL JOIN Passenger
WHERE RouteTo = 'New York' AND City="Los Angeles";
```

in the three following scenarios:

1. The query runs without any additional indexes on the tables. (3 points)
2. Both tables have secondary hash indexes: **Ticket**, built on **PassengerId** (4K blocks), with no overflow chains. **Passenger**, built on **City** (600 blocks), with no overflow chains. (4 points)
3. In addition to the two previous indexes, **Ticket** has a composite B+ tree index built on (**RouteTo**, **PassengerId**) with 3 levels, and 6K leaf nodes. (4 points)

For each scenario,

- Briefly explain the plan (the join method, scan&look up, etc.).
- Clearly explain the number of blocks/leaf nodes that need to be retrieved; report all the needed computations to understand the total cost.
- Write the complete formula of the cost.

Solution.

1.
 - (NLJ: $\text{Cost} = 2 + 2,5K + 2,5K * 45K = 112,5M$ I/O accesses)
 - Scan ticket, filter on **RouteTo**, lookup **Passenger** (given the **passengerId** of the **Ticket**, use the B+ to search the unique **Passenger** tuple):
 $\text{Cost} = 45K + 300K/250 * 3 = 45K + 1200*3 = \mathbf{48,6K}$ I/O accesses
 - Scan **Passenger**, filter on **City**, lookup **Ticket** (with a full scan):
 $\text{Cost} = 2 + 2,5K + 40K/500 * 45K = 2 + 2,5K + 80 * 45K = 3,6M$ I/O accesses
2. Use $h(\text{City}="Los Angeles")$ and access the hash index, follow the pointers to access **Passengers** from LA, lookup **Ticket** through $h(\text{PassengerID})$
 $\text{Cost} = 1 + 40K/500$ (passengers from LA) + $40K/500 * (1 + 300K \text{ tickets}/40K \text{ passengers}) = 1 + 80 + 80 * (1 + 7,5) = \mathbf{761}$ I/O accesses (if 7,5 is rounded to 8 the result is 801)
3. In the new B+ index, use $\text{routeTo}="New York"$ and access all the leaf nodes with $\text{routeTo}="New York"$: this requires 2 intermediate levels + $(300K \text{ tickets}/250 \text{ routeTo})/(300K \text{ ticket}/6K \text{ leaf nodes}) = 24$ leaf nodes – here there are the **PassengerIDs**, no need to access the full tuples!
The number of passengers with destination New York are $300K/250 = 1200$.
Use the **PassengerIDs** to retrieve the **Passenger** tuples and check the **City**. For each **passengerID**: 2 intermediate nodes + 1 leaf node in the B+ primary structure because **PassengerID** is unique in the table.
 $\text{Cost} = 2 + (24 \text{ or } 25) + 1200 * (2 + 1) = 3626 \text{ or } \mathbf{3627}$ I/O accesses

Alternative plan: retrieve the passengers from Los Angeles using the hash index on **City** and following the 80 pointers; now, you have 80 **PassIDs** to be combined with $\text{routeTo}="New York"$ in the search in the B+: this search is done for the 80 passengers and the cost is the number of levels of the B+ (the number of tickets of a single passenger is 7.5, those for NY are less and can be contained in a single leaf node).

$\text{Cost} = 1 + 80 + 80 * 3 = 321$ I/O accesses